

CRYSTAL DOMAIN · PYTHON FOR DATA SCIENCE

Pandas

Topic-wise Explanation + Practice

36 topics · Concept → Explanation → Syntax → Example → Output → Practice Task

Sreenivas

Assistant Professor, CSE (AI & ML)

Mentor, Crystal Domain — Materials Informatics

How to Use This Guide

Every topic follows the same six-step format, so students always know what to expect:

Step	What it contains
1. Concept	What the feature is and why it matters
2. Explanation	Plain-language description with analogies
3. Syntax	The exact code pattern to remember
4. Practice Example	A small runnable example
5. Expected Output	What the student should see on screen
6. Practice Task	A short exercise to try independently

Teaching tip: cover these topics one at a time rather than all together. Each topic works well as a 5–10 minute explanation followed immediately by a coding exercise, then a larger practice problem at the end of the session.

Contents

Part	Theme	Topics
Part 1	Foundations	1–3
Part 2	Loading and Inspecting Data	4–12
Part 3	Selecting Data	13–16
Part 4	Filtering and Sorting	17–19
Part 5	Creating New Columns	20–21
Part 6	Cleaning Data	22–25
Part 7	Exploring and Summarising Data	26–29
Part 8	Combining DataFrames	30–31
Part 9	Text, Dates and Exporting	32–34
Part 10	Pandas for Machine Learning	35–36

Part 1 — Foundations

1. What is Pandas?

EXPLANATION

Pandas is a Python library used to work with structured / tabular data. Think of a table like Excel:

Name	Marks	Attendance
Ravi	85	92
Priya	78	88
Arun	65	72

Pandas can store this entire table as a **DataFrame**.

SYNTAX

```
import pandas as pd
```

`pd` is the commonly used alias for Pandas.

PRACTICE EXAMPLE

```
import pandas as pd

data = {
    "Name": ["Ravi", "Priya", "Arun"],
    "Marks": [85, 78, 65],
    "Attendance": [92, 88, 72]
}

df = pd.DataFrame(data)

print(df)
```

EXPECTED OUTPUT

	Name	Marks	Attendance
0	Ravi	85	92
1	Priya	78	88
2	Arun	65	72

PRACTICE TASK

Create a DataFrame containing 5 student names, roll numbers, marks and attendance.

2. Pandas Series

EXPLANATION

A **Series** is a one-dimensional data structure. Think of it as **one column of an Excel sheet**.

PRACTICE EXAMPLE

```
import pandas as pd

marks = pd.Series([85, 78, 65, 91])

print(marks)
```

EXPECTED OUTPUT

```
0    85
1    78
2    65
3    91
dtype: int64
```

The numbers on the left are the **index**.

Practice with a custom index

```
marks = pd.Series(
    [85, 78, 65, 91],
    index=["Ravi", "Priya", "Arun", "Sita"]
)

print(marks)
```

```
Ravi    85
Priya   78
Arun    65
Sita    91
```

Now access a single value by its label:

```
print(marks["Ravi"])
```

```
85
```

PRACTICE TASK

Create a Series with Java = 85, Python = 92, DBMS = 78, ML = 88.

Print the marks obtained in Python.

3. DataFrame

EXPLANATION

A **DataFrame** is a two-dimensional table consisting of rows, columns, an index and the data itself. It is the most important Pandas structure.

PRACTICE EXAMPLE

```
import pandas as pd

data = {
    "Name": ["Ravi", "Priya", "Arun", "Sita"],
    "Marks": [85, 78, 65, 91],
    "Attendance": [92, 88, 72, 95]
}

df = pd.DataFrame(data)

print(df)
```

VISUAL UNDERSTANDING

	Columns ↓		
	Name	Marks	Attendance
Row 0	Ravi	85	92
Row 1	Priya	78	88
Row 2	Arun	65	72
Row 3	Sita	91	95

PRACTICE TASK

Create a DataFrame with the columns RollNo, Name, Department and Marks for 5 students.

Part 2 — Loading and Inspecting Data

4. Reading a CSV File

EXPLANATION

Most real-world datasets are stored as CSV files, for example `students.csv`. Pandas can load one into a DataFrame.

PRACTICE EXAMPLE

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df)
```

Key idea: CSV File → `pd.read_csv()` → DataFrame → Analysis

PRACTICE TASK

Create a file called `students.csv` with the content below, then load it using Pandas.

```
Name,Marks,Department
Ravi,85,CSE
Priya,78,AI ML
Arun,65,CSE
Sita,91,AI ML
```

5. Reading Excel Files

EXPLANATION

Pandas can also read Excel files.

PRACTICE EXAMPLE

```
import pandas as pd

df = pd.read_excel("students.xlsx")

print(df)
```

To read a specific sheet:

```
df = pd.read_excel(
    "students.xlsx",
    sheet_name="Sheet1"
)
```

PRACTICE TASK

Create an Excel file containing student data and load it into Pandas.

6. head()

EXPLANATION

When a dataset contains thousands of rows, we do not want to print everything. `head()` displays the **first 5 rows** by default.

```
print(df.head())      # first 5 rows
print(df.head(10))    # first 10 rows
```

PRACTICE TASK

Load a dataset and display the first 5, first 10 and first 20 rows.

7. tail()

EXPLANATION

`tail()` displays the **last rows** of the dataset.

```
print(df.tail())      # last 5 rows
print(df.tail(10))    # last 10 rows
```

PRACTICE TASK

Display the last 5 and last 10 records of your dataset.

8. shape

EXPLANATION

`shape` tells us (number of rows, number of columns).

```
print(df.shape)
```

EXPECTED OUTPUT

```
(1000, 15)
```

Meaning: **1000 rows** and **15 columns**.

PRACTICE TASK

Print the shape of your dataset and write down the number of rows and the number of columns.

9. columns

EXPLANATION

`columns` displays all column names.

```
print(df.columns)
```

```
Index(['Name', 'Marks', 'Department', 'Attendance'], dtype='object')
```

PRACTICE TASK

Print all column names from your dataset.

10. dtypes

EXPLANATION

Every column has a data type.

```
print(df.dtypes)
```

```
Name      object
Marks      int64
Percentage float64
Passed      bool
```

PRACTICE TASK

Which columns contain integers?

Which columns contain decimals?

Which columns contain text?

11. info()

EXPLANATION

`info()` gives a quick overview of the dataset. It shows column names, number of rows, non-null values, data types and memory usage.

```
df.info()
```

PRACTICE TASK

Run `info()` and identify any columns containing missing values.

12. describe()

EXPLANATION

`describe()` provides statistical information for numerical columns: count, mean, std, min, 25%, 50%, 75% and max.

```
df.describe()
```

For example, for the Marks column: mean = 76.5, min = 35, max = 95.

PRACTICE TASK

Use `describe()` to determine:

1. Average marks
2. Minimum marks
3. Maximum marks
4. Median marks (the 50% row)

Part 3 — Selecting Data

13. Selecting a Column

EXPLANATION

Use square brackets with the column name to select one column.

```
marks = df["Marks"]  
  
print(marks)
```

You can also write `df.Marks`, but `df["Marks"]` is generally preferred (it works with any column name, including those with spaces).

PRACTICE TASK

Select Name, Marks and Attendance, one at a time.

14. Selecting Multiple Columns

EXPLANATION

Pass a **list** of column names (note the double square brackets).

```
result = df[["Name", "Marks"]]  
  
print(result)
```

EXPECTED OUTPUT

	Name	Marks
0	Ravi	85
1	Priya	78
2	Arun	65

PRACTICE TASK

Display only Name, Department and Marks.

15. Selecting Rows with `iloc`

EXPLANATION

`iloc` means **integer-location** based selection. It works using row / column **positions**.

```
df.iloc[0]      # first row
df.iloc[2]      # third row
df.iloc[0:3]    # first three rows
df.iloc[0, 1]   # row position 0, column position 1
```

PRACTICE TASK

Using `iloc`:

1. Print the first row.
2. Print the third row.
3. Print the first 3 rows.
4. Print the marks of the first student.

16. Selecting Rows with `loc`

EXPLANATION

`loc` selects data using **labels**.

```
df.loc[0]      # row with label 0
df.loc[0, "Marks"] # row label 0, column named Marks
```

`iloc` vs `loc`: `iloc` → position, `loc` → label.

PRACTICE TASK

Use `loc` to print the name, marks and attendance of the first student.

Part 4 — Filtering and Sorting

17. Filtering Data

This is one of the most important Pandas topics.

EXPLANATION

Suppose we want only students scoring more than 80.

```
df[df["Marks"] > 80]
```

Pandas first creates a condition (True / False for every row), then returns only the rows where the condition is **True**:

```
85 > 80 → True
78 > 80 → False
65 > 80 → False
91 > 80 → True
```

PRACTICE TASK

Find students with:

1. Marks > 80
2. Marks < 50
3. Attendance > 90
4. Attendance < 75

18. Multiple Conditions

AND — use &

```
df[
    (df["Marks"] > 80) &
    (df["Attendance"] > 90)
]
```

Meaning: marks greater than 80 **AND** attendance greater than 90.

OR — use |

```
df[
    (df["Marks"] > 90) |
    (df["Attendance"] > 95)
]
```

Meaning: marks greater than 90 **OR** attendance greater than 95.

Remember: wrap each condition in its own parentheses.

PRACTICE TASK

Find students who have Marks ≥ 80 AND Attendance ≥ 85 .

19. Sorting

EXPLANATION

`sort_values()` sorts rows according to a column.

```
df.sort_values("Marks")                # ascending
df.sort_values("Marks", ascending=False) # descending
```

PRACTICE TASK

Sort your student dataset:

1. Lowest $\rightarrow$ highest marks
2. Highest $\rightarrow$ lowest marks
3. Highest attendance first

Part 5 — Creating New Columns

20. Adding a New Column

EXPLANATION

We can create a new column easily — either with a fixed value or a calculation.

```
df["Result"] = "Pass"          # every row gets "Pass"

df["BonusMarks"] = df["Marks"] + 5
```

PRACTICE TASK

Create a BonusMarks column equal to Marks + 5.

21. Conditional Column

EXPLANATION

Suppose Marks ≥ 40 means Pass and Marks < 40 means Fail.

```
df["Result"] = df["Marks"].apply(
    lambda x: "Pass" if x >= 40 else "Fail"
)
```

PRACTICE TASK

Create a Grade column:

Marks	Grade
90 and above	A+
80 – 89	A
70 – 79	B
60 – 69	C
50 – 59	D
Below 50	F

Part 6 — Cleaning Data

22. Missing Values

EXPLANATION

Real datasets often contain missing values. Pandas represents missing numerical data as **NaN**.

Name	Marks
Ravi	85
Priya	NaN
Arun	65

Detect and count missing values

```
df.isnull()          # True where a value is missing
df.isnull().sum()    # count per column
```

PRACTICE TASK

Create a DataFrame with some missing marks and find how many values are missing in each column.

23. dropna()

EXPLANATION

Removes rows containing missing values.

```
df = df.dropna()

# only consider one column
df = df.dropna(subset=["Marks"])
```

PRACTICE TASK

Remove all students whose marks are missing.

24. fillna()

EXPLANATION

Instead of deleting missing data, we can replace it.

Replace with zero

```
df["Marks"] = df["Marks"].fillna(0)
```

Replace with the average

```
df["Marks"] = df["Marks"].fillna(  
    df["Marks"].mean()  
)
```

PRACTICE TASK

Create missing attendance values and replace them with the average attendance.

25. Duplicate Data

EXPLANATION

Datasets can contain duplicate records.

```
df.duplicated()          # check  
df.duplicated().sum()    # count  
df = df.drop_duplicates() # remove
```

PRACTICE TASK

Create a duplicate student record and remove it.

Part 7 — Exploring and Summarising Data

26. unique()

EXPLANATION

Finds all the different values in a column.

```
df["Department"].unique()
```

```
['CSE', 'AIML', 'ECE']
```

PRACTICE TASK

Find all unique departments in your dataset.

27. value_counts()

EXPLANATION

Counts how many times each value occurs.

```
df["Department"].value_counts()
```

CSE	40
AIML	30
ECE	20

ML tip: `value_counts()` is especially useful for checking the **class distribution** of a target column.

PRACTICE TASK

Find how many students belong to each department.

28. groupby()

EXPLANATION

`groupby()` divides data into groups and lets us calculate statistics for each group.

```
df.groupby("Department")["Marks"].mean()
```

This calculates the **average marks for each department**.

PRACTICE TASK

Calculate department-wise:

- Average marks
- Maximum marks
- Minimum marks
- Number of students

29. agg()

EXPLANATION

`agg()` performs multiple calculations at once.

```
df.groupby("Department")["Marks"].agg(  
    ["mean", "min", "max", "count"]  
)
```

PRACTICE TASK

For each department calculate the mean, minimum, maximum and count of marks.

Part 8 — Combining DataFrames

30. merge()

EXPLANATION

`merge()` combines two DataFrames using a common column.

students

RollNo	Name
101	Ravi
102	Priya
103	Arun

marks

RollNo	Marks
101	85
102	78
103	65

PRACTICE EXAMPLE

```
result = pd.merge(
    students,
    marks,
    on="RollNo"
)

print(result)
```

EXPECTED OUTPUT

	RollNo	Name	Marks
0	101	Ravi	85
1	102	Priya	78
2	103	Arun	65

PRACTICE TASK

Create students and attendance DataFrames and merge them using RollNo.

31. concat()

EXPLANATION

`concat()` combines DataFrames vertically (stacking rows) or horizontally.

```
result = pd.concat([df1, df2])
```

```
df1: Ravi, Priya  
df2: Arun, Sita
```

↓ concat

```
Ravi  
Priya  
Arun  
Sita
```

PRACTICE TASK

Create two DataFrames, Section A and Section B, and combine them.

Part 9 — Text, Dates and Exporting

32. String Operations

EXPLANATION

Pandas provides the `.str` accessor for text processing.

```
df["Name"].str.upper()          # uppercase
df["Name"].str.lower()         # lowercase
df[df["Name"].str.contains("a")] # names containing "a"
```

PRACTICE TASK

For a Name column:

1. Convert all names to uppercase.
2. Convert all names to lowercase.
3. Find names containing "an".

33. Date Handling

EXPLANATION

Pandas can convert text dates into real date objects.

```
df["Date"] = pd.to_datetime(df["Date"])

df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
```

PRACTICE TASK

Create a date column and extract the Year, Month and Day.

34. Exporting Data

EXPLANATION

After processing data, we often need to save it.

```
df.to_csv("processed_data.csv", index=False)

df.to_excel("processed_data.xlsx", index=False)
```

PRACTICE TASK

Take your cleaned student DataFrame and save it as `students_cleaned.csv`.

Part 10 — Pandas for Machine Learning

35. Pandas for Machine Learning

This is the most important section for the **Crystal Domain / Materials Science** projects. Suppose you have a materials dataset with these columns:

```
material_id, formula, volume, density, formation_energy, band_gap
```

Load the data

```
df = pd.read_csv("materials.csv")
```

Step 1 — Inspect

```
df.head()
df.shape
df.info()
df.describe()
```

Step 2 — Check missing data

```
df.isnull().sum()
```

Step 3 — Clean

```
df = df.dropna(subset=["band_gap"])
```

Step 4 — Select features

```
X = df[
    [
        "volume",
        "density",
        "formation_energy"
    ]
]
```

Step 5 — Select target

```
y = df["band_gap"]
```

THE PIPELINE

```
Pandas → Clean Dataset → X, y → Scikit-learn
                                   ↓
                               ML Algorithm
                                   ↓
                               Prediction
```

36. Complete Mini Practice Project

After learning all the individual topics, give students this exercise.

DATASET

```
import pandas as pd

data = {
    "RollNo": [101, 102, 103, 104, 105, 106],
    "Name": [
        "Ravi", "Priya", "Arun",
        "Sita", "Kiran", "Anu"
    ],
    "Department": [
        "CSE", "AIML", "CSE",
        "AIML", "ECE", "CSE"
    ],
    "Marks": [
        85, 78, 35, 91, 67, 88
    ],
    "Attendance": [
        92, 88, 65, 95, 76, 90
    ]
}

df = pd.DataFrame(data)
```

STUDENTS MUST PERFORM THESE TASKS

#	Task	Hint
1	Display the first 3 rows	<code>df.head(3)</code>
2	Find the number of rows and columns	<code>df.shape</code>
3	Display column names	<code>df.columns</code>
4	Display statistical information	<code>df.describe()</code>
5	Display only Name and Marks	
6	Find students with marks above 80	
7	Find students with attendance below 75	
8	Sort students by marks, highest to lowest	
9	Create a Result column (Marks $\geq 40 \rightarrow$ Pass, else Fail)	

#	Task	Hint
10	Find average marks department-wise	
11	Find the number of students in each department	
12	Save the final DataFrame as student_analysis.csv	

The Complete Pandas Learning Flow

Follow this sequence from first principles to a real machine-learning dataset.

- | | | | |
|----|---------------------------|----|-----------------------|
| 1 | Pandas Introduction | 11 | Duplicates |
| 2 | Series | 12 | String Operations |
| 3 | DataFrame | 13 | GroupBy & Aggregation |
| 4 | Reading CSV / Excel | 14 | Merge & Concat |
| 5 | Dataset Inspection | 15 | Date Handling |
| 6 | Selecting Rows & Columns | 16 | Exporting Data |
| 7 | Filtering | 17 | Pandas + NumPy |
| 8 | Sorting | 18 | Pandas + Scikit-learn |
| 9 | Adding / Removing Columns | 19 | Real ML Dataset |
| 10 | Missing Values | | |